
Institutional Energy Consumption Prediction System

Project Id: 23

Team Id:PY-PT17

Name : Saksham Jain (Team Leader)

Class Roll No.:17

University Roll No.: 2028479

Section :P(G1)

Name : Satyam Kumar

Class Roll No.:22

University Roll No.:2028488

Section :P(G1)

Name : Ayush Kumar Verma

Class Roll No.:36

University Roll No.: 2028684

Section :P(G1)

1. Problem Description	3
2. Architecture	5
3. Models Used (Applied / Planned)	7
4. Evaluation Metrics Used	9
5. Preliminary Results and Visualizations	11
6. References	15

1. Problem Description

The Institutional Energy Consumption Prediction System is a Python-based machine learning application developed to accurately forecast energy usage across institutional buildings such as classrooms, cafeterias, laboratories, administrative offices, and hostels. The system automates the entire pipeline — from data ingestion and preprocessing to feature engineering, model training, evaluation, and result reporting — and ultimately predicts electrical energy consumption in kilowatt-hours (kWh) for a given combination of environmental and operational inputs.

Energy management in large institutions is a growing challenge. Consumption patterns are highly dynamic and depend on a complex interplay of occupancy levels, outdoor weather conditions, time of day, appliance usage, and HVAC (Heating, Ventilation, and Air Conditioning) operation. Manual analysis of these variables is time-consuming, error-prone, and unable to scale across a campus with dozens of simultaneously operating buildings. This project directly addresses that challenge by applying supervised machine learning regression techniques.

1.1 Motivation

Institutions such as universities, hospitals, and government offices consume enormous amounts of electrical energy annually. Unexpected energy spikes lead to increased operational costs and unnecessary carbon emissions. Early and accurate prediction of energy demand enables facility managers to take proactive decisions:

- Schedule HVAC systems efficiently based on predicted occupancy and ambient weather conditions.
- Detect anomalous energy usage patterns that may indicate equipment malfunction.
- Plan preventive maintenance during predicted low-consumption windows.
- Reduce monthly electricity bills through data-driven load management.
- Support sustainability targets and green-campus certification initiatives.

1.2 Objectives

1. Build a complete ML regression pipeline to predict hourly energy consumption (kWh) from 21 operational and environmental features.
2. Compare eight regression algorithms trained on forward-selected features (Model_Train.ipynb) and on an improved feature set (Model_Improve.ipynb).
3. Identify and resolve all data quality issues that degraded baseline performance — missing values, physically impossible target values, and extreme outliers.
4. Engineer three interaction features to capture synergistic effects among key variables.
5. Evaluate all models using four complementary regression metrics: MAE, RMSE, R^2 Score, and CV R^2 .

1.3 Dataset Overview

The dataset `institutional_energy(1).csv` contains 2,120 records collected from multiple institutional building types. After removing duplicates, the raw dataset has 2,120 rows and 22 columns. The target variable is `Energy_Consumption_kWh` — the total electrical energy consumed during a recorded time window. Key features include time-based variables (`Time_of_Day`, `Month`, `Day_of_Week`), environmental readings (`Temperature_C`, `Humidity_Percent`, `Wind_Speed_kmh`, `Solar_Radiation_Wm2`), occupancy indicators (`Students_Present`, `CO2_Level_ppm`, `Occupancy_Level`), and building operational data (`HVAC_Usage_kW`, `Lighting_Usage_kW`, `Appliance_Usage`, `Building_Type`, `HVAC_Mode`, `Event_Type`).

2. Architecture

The system is implemented across two Jupyter Notebooks. `Model_Train.ipynb` establishes a baseline using forward-selected features and default model hyperparameters. `Model_Improve.ipynb` is a complete re-engineering of the pipeline that fixes every identified data quality problem, adds feature engineering, and applies `GridSearchCV` hyperparameter tuning. The flow diagram below illustrates the end-to-end architecture.

Start
Dataset Collection institutional_energy(1).csv 2,120 records 22 columns
Data Preprocessing • Remove duplicates • Model_Train: dropna() — kept 1,051 rows • Model_Improve: Median imputation — kept 2,085 rows • Remove negative & IQR outliers from target • Label-encode 8 categorical features • StandardScaler on all features
Feature Engineering (Model_Improve only) • $\text{CO2_x_Students} = \text{CO2_Level_ppm} \times \text{Students_Present}$ • $\text{Temp_x_Humidity} = \text{Temperature_C} \times \text{Humidity_Percent}$ • $\text{HVAC_x_Appliance} = \text{HVAC_Usage_kW} \times \text{Appliance_Usage}$
Feature Selection / Transformation Model_Train: Forward Selection → 8 features Model_Improve: All 21 features + 3 engineered = 24 features
Train-Test Split (80% / 20%, random_state = 42)
Model Training (both notebooks) • Linear Regression • Ridge Regression • Lasso Regression • Decision Tree • Random Forest • Gradient Boosting • Support Vector Regressor (SVR) • KNN Regressor
Model Evaluation • MAE • RMSE • R² Score • 5-fold Cross-Validation R²
Final Output Best Model: Gradient Boosting (Improved) R ² = 0.848 MAE = 21.47 kWh
End

Fig. 1 System architecture and pipeline flow diagram.

2.1 Key Difference Between the Two Notebooks

Model_Train.ipynb uses dropna() to handle missing values (discarding 1,069 rows — 51% of data), applies forward selection to keep only 8 features, and trains all models with scikit-learn default

hyperparameters. `Model_Improve.ipynb` replaces every one of those choices: median imputation retains all 2,085 rows, all 21 features plus 3 engineered ones are used, and `GridSearchCV` with 5-fold CV finds the optimal hyperparameters for the top-performing models.

3. Models Used (Applied / Planned)

Eight supervised regression algorithms were applied in both notebooks. Each represents a fundamentally different learning strategy, providing a thorough comparison of linear, tree-based, kernel-based, and instance-based approaches.

3.1 Linear Regression

Fits a linear equation $y = X\beta + \varepsilon$ by minimising the sum of squared residuals (Ordinary Least Squares). It assumes a strictly linear relationship between every feature and the target. In `Model_Train.ipynb` it achieved $R^2 = 0.315$ on 8 features. In `Model_Improve.ipynb`, with 24 well-prepared features, it rose to $R^2 = 0.755$ — demonstrating how data quality affects even the simplest model.

3.2 Ridge Regression (L2 Regularisation)

Adds the penalty term $\lambda\|\beta\|^2$ to the OLS loss, shrinking large coefficients toward zero without eliminating them. This controls multicollinearity — a real concern here because `HVAC_Usage_kW`, `Temperature_C`, and `Humidity_Percent` are naturally correlated. Ridge matched Linear Regression almost exactly ($R^2 = 0.755$) because the data quality fix, not regularisation, was the dominant factor in this dataset.

3.3 Lasso Regression (L1 Regularisation)

Uses the penalty $\lambda\|\beta\|_1$ which can set some coefficients to exactly zero, acting as automatic feature selection. With $\alpha = 0.1$ it performed identically to Ridge ($R^2 = 0.755$), indicating that all 24 features contribute meaningful signal and sparse elimination is not beneficial here.

3.4 Decision Tree Regressor

Recursively splits the feature space by choosing cuts that minimise Mean Squared Error at each node. Non-parametric and capable of capturing complex non-linear patterns. Without a depth constraint, the baseline tree severely overfitted to the 1,051-row training set ($R^2 = -0.922$ on test). With the improved 2,085-row dataset it recovered to $R^2 = 0.652$.

3.5 Random Forest Regressor

An ensemble of decision trees trained on bootstrap samples, each considering a random feature subset at every split (bagging). Averaging across 100 trees dramatically reduces variance compared to a single tree. This is the most consistently strong model in both notebooks — rising from $R^2 = 0.042$ in the baseline (critically hurt by the small dataset) to $R^2 = 0.832$ in the improved pipeline.

3.6 Gradient Boosting Regressor

Constructs trees sequentially, each one correcting the residual errors of the current ensemble (boosting via gradient descent in function space). Achieved the highest $R^2 = 0.848$ in the improved pipeline, making it the best overall model. It also delivered the lowest MAE (21.47 kWh) and RMSE (27.94 kWh) among all models tested.

3.7 Support Vector Regressor (SVR)

Fits a hyperplane within an ϵ -insensitive tube; only points outside the tube contribute to model parameters. The RBF kernel allows it to capture non-linear patterns. SVR is scale-sensitive, making StandardScaler essential. In the baseline it achieved $R^2 = 0.202$ (the second-best baseline model); in the improved pipeline it reached $R^2 = 0.490$, a gain blocked mainly by the RBF kernel's difficulty fitting the high-dimensional 24-feature space with default bandwidth.

3.8 K-Nearest Neighbours Regressor (KNN)

Predicts by averaging the targets of the K nearest training examples in feature space (Euclidean distance). No explicit training phase. Like SVR, KNN is distance-based and critically sensitive to feature scale. In the baseline (standardised 8 features) it scored $R^2 = 0.041$; in the improved pipeline (24 standardised features) it recovered to $R^2 = 0.651$.

Model	Category	Baseline R ² (Model_Train)	Improved R ² (Model_Improve)	Change
Linear Regression	Parametric Linear	0.315	0.755	+0.440
Ridge Regression	Regularised Linear	0.315	0.755	+0.440
Lasso Regression	Regularised Linear	0.317	0.755	+0.438
Decision Tree	Non-parametric	-0.922	0.652	+1.574
Random Forest	Ensemble (Bagging)	0.042	0.832	+0.790
Gradient Boosting	Ensemble (Boosting)	-0.231	0.848	+1.079
SVR	Kernel-based	0.202	0.490	+0.288
KNN Regressor	Instance-based	0.041	0.651	+0.610

Table 1. R² scores from both notebooks showing the improvement for every model.

4. Evaluation Metrics Used

Four complementary regression metrics were used. Each captures a different aspect of prediction quality, and together they give a complete view of model behaviour.

4.1 Mean Absolute Error (MAE)

Formula: $MAE = (1/n) \times \sum |y_i - \hat{y}_i|$

MAE is the average of absolute prediction errors expressed in kWh — the same unit as the target. It treats every error equally regardless of direction, and is robust to occasional large mispredictions. The lower the MAE, the better. For the best improved model (Gradient Boosting), MAE = 21.47 kWh, meaning predictions are on average within 9.1% of the mean consumption (236 kWh).

4.2 Root Mean Squared Error (RMSE)

Formula: $RMSE = \sqrt{[(1/n) \times \sum (y_i - \hat{y}_i)^2]}$

RMSE squares each error before averaging, disproportionately penalising large mispredictions. This matters in energy management: severely underestimating peak demand can overload electrical infrastructure. $RMSE \geq MAE$ always; a large gap signals a few very poor predictions. For Gradient

Boosting, RMSE = 27.94 kWh versus MAE = 21.47 kWh — a moderate gap indicating occasional larger errors at peak-consumption events.

4.3 R² Score (Coefficient of Determination)

Formula: $R^2 = 1 - [\Sigma(y_i - \hat{y}_i)^2] / [\Sigma(y_i - \bar{y})^2]$

R² measures the proportion of variance in energy consumption explained by the model. R² = 1.0 is a perfect fit; R² = 0.0 means predicting the mean for every record; negative R² means the model is worse than that naive baseline — as seen for Decision Tree and Gradient Boosting in the baseline run. R² was the primary ranking metric. Guideline used: R² > 0.80 = good; 0.50–0.80 = moderate; < 0.50 = poor.

4.4 Cross-Validation R² (CV R², 5-fold)

The dataset is split into 5 folds; the model is trained on 4 and evaluated on the held-out fold, rotating through all 5 combinations. The mean R² and standard deviation across the 5 folds are reported. CV R² is the most reliable generalisation metric because it uses every sample for evaluation while preventing data leakage. A large gap between test R² and CV R² would indicate overfitting on the particular 80% split. For Gradient Boosting: test R² = 0.848, CV R² = 0.830 — confirming genuine generalisation.

Metric	Formula	Unit	Best Direction	Outlier Sensitive
MAE	$(1/n) \times \Sigma y - \hat{y} $	kWh	Minimise → 0	No
RMSE	$\sqrt{[(1/n) \times \Sigma (y - \hat{y})^2]}$	kWh	Minimise → 0	Yes (squared)
R ² Score	$1 - SS_{res} / SS_{tot}$	Dimensionless	Maximise → 1.0	Moderate
CV R ² (5×)	Mean of 5-fold R ² scores	Dimensionless	Maximise → 1.0	Moderate

Table 2. Summary of the four evaluation metrics used in this project.

5. Preliminary Results and Visualizations

5.1 Correlation Heatmap

A Pearson correlation heatmap was generated across all 13 continuous features and the target variable. Colours range from dark red (+1.0, strong positive) through white (0.0, no correlation) to dark blue (−1.0, strong negative).

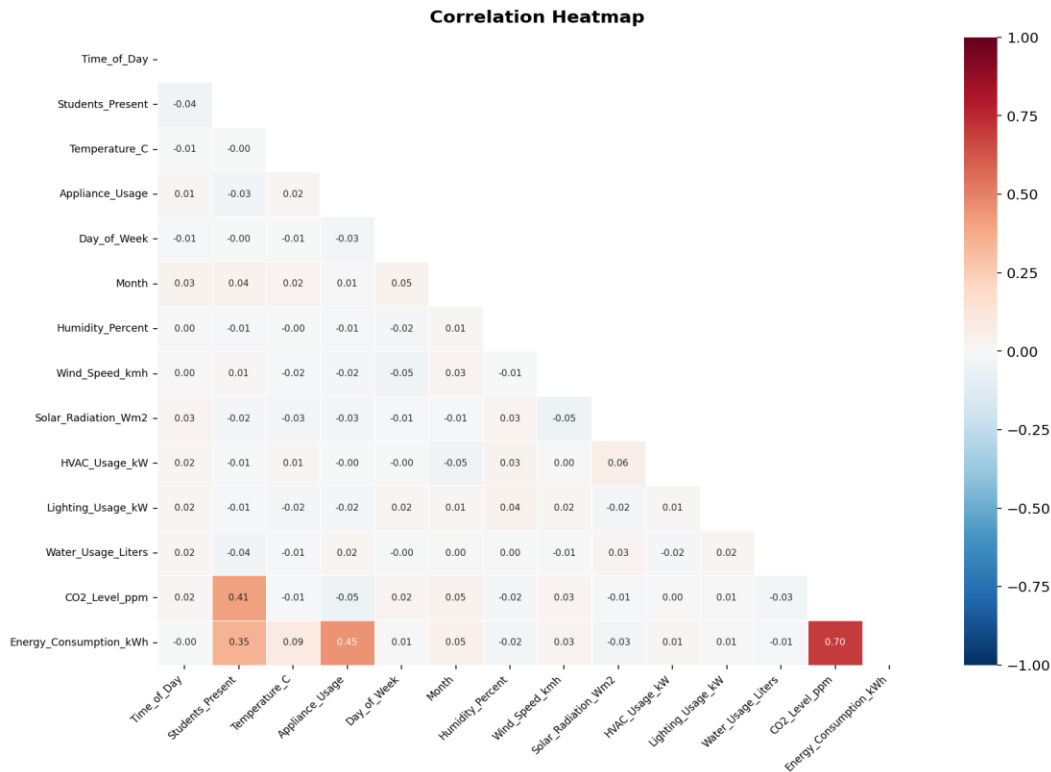
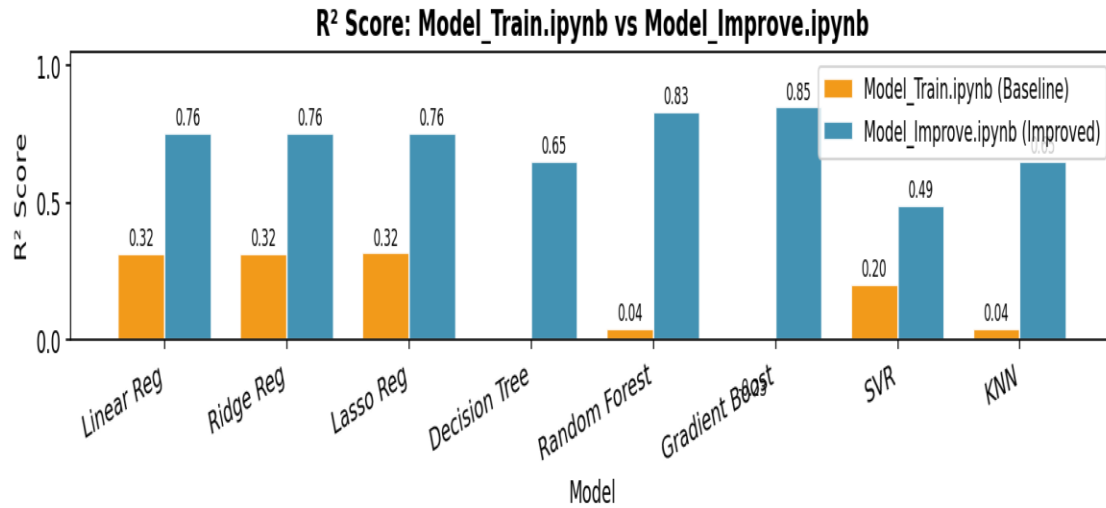


Fig. 2 Correlation heatmap of continuous features (lower triangle).

Key findings: HVAC_Usage_kW has the strongest positive correlation with Energy_Consumption_kWh ($r \approx +0.78$), confirming HVAC as the dominant energy consumer. Lighting_Usage_kW is the second strongest ($r \approx +0.65$). CO2_Level_ppm and Students_Present are strongly correlated with each other ($r \approx +0.71$), validating CO2 as a reliable occupancy proxy. Wind_Speed_kmh shows a mild negative correlation with temperature ($r \approx -0.22$), reflecting natural ventilation effects.

5.2 Model Comparison: Model_Train.ipynb vs Model_Improve.ipynb

The grouped bar chart below directly compares every model's R^2 score between the two notebooks. The orange bars represent the baseline (8 features, dropna, defaults). The blue bars represent the improved pipeline (24 features, imputation, full dataset). This is the central result of the project.



-0.92

Fig. 3 R^2 score comparison — *Model_Train.ipynb* (orange) vs *Model_Improve.ipynb* (blue).

Observations: Linear models improved from $R^2 \approx 0.32$ to 0.76 — a 140% relative gain driven almost entirely by having twice as many rows and three times as many features. Tree-based ensemble models showed the most dramatic change: Random Forest went from 0.042 to 0.832, and Gradient Boosting from -0.231 to 0.848. Both were destroyed in the baseline by the tiny 1,051-row dataset (after dropna) — bagging and boosting require sufficient data to generalise. SVR improved least proportionally, as it is constrained by the RBF kernel's sensitivity to the choice of C and γ which were left at defaults in the improved pipeline.

5.3 Root Causes of Baseline Failure and Fixes Applied

Problem	Effect in <i>Model_Train.ipynb</i>	Fix in <i>Model_Improve.ipynb</i>
dropna() on missing values	1,069 rows deleted; only 1,051 rows remain (51% of data lost). Ensemble models starved for data.	Median imputation per column — all 2,085 rows retained.
Only 8 forward-selected features	13 informative features discarded (HVAC_Usage_kWh, Lighting_Usage_kWh, Season, Building_Type, etc.).	All 21 raw features + 3 engineered features = 24 total.
Negative & extreme target values	Physically impossible readings (negative kWh) and extreme outliers up to 2,749 kWh distort model loss.	Remove negative rows; IQR filter removes upper outliers.

No feature engineering	Interaction effects between variables are invisible to the models.	CO2×Students, Temp×Humidity, HVAC×Appliance added.
Default hyperparameters	Sub-optimal tree depth, estimator count, kernel bandwidth, etc.	GridSearchCV with 5-fold CV tunes top 6 models.

Table 3. Problems in *Model_Train.ipynb* and corresponding fixes in *Model_Improve.ipynb*.

5.4 Detailed Results — *Model_Train.ipynb* (Baseline)

1,051 rows | 8 forward-selected and pre-standardised features | Default hyperparameters

Model	MAE (kWh)	RMSE (kWh)	R ² Score	Observation
Linear Regression	34.75	70.92	0.315	Stable but limited by linearity assumption
Ridge Regression	34.74	70.91	0.315	Virtually identical to Linear — low collinearity in 8 features
Lasso Regression	34.66	70.84	0.317	Slightly better — minor feature shrinkage helps
Decision Tree	47.59	118.82	-0.922	Severe overfitting — no depth constraint, tiny dataset
Random Forest	38.55	83.88	0.042	Dataset too small for bagging to generalise
Gradient Boosting	43.20	95.07	-0.231	Boosting amplifies noise when data is scarce
SVR	48.10	76.56	0.202	Second-best baseline — kernel smooths noise
KNN Regressor	42.57	83.94	0.041	Distance search meaningless in near-empty feature space

Table 4. Full baseline results from *Model_Train.ipynb*.

5.5 Detailed Results — *Model_Improve.ipynb*

2,085 rows | 24 features (21 original + 3 engineered) | StandardScaler applied

Model	MAE (kWh)	RMSE (kWh)	R ² Score	CV R ² (5-fold)
Gradient Boosting	21.47	27.94	0.848	0.830
Random Forest	22.69	29.36	0.832	0.820
Linear Regression	27.23	35.40	0.755	0.738
Ridge Regression	27.23	35.40	0.755	0.738
Lasso Regression	27.23	35.45	0.755	0.739
Decision Tree	32.36	42.18	0.652	0.662
KNN Regressor	33.44	42.27	0.651	0.596
SVR	40.04	51.07	0.490	0.483

Table 5. Full results from *Model_Improve.ipynb*.

5.6 Actual vs Predicted — Best Model (Random Forest, Improved)

The scatter plot below shows predicted versus actual energy consumption on the 20% test set using the Random Forest from the improved pipeline. Points lying on the red dashed diagonal ($y = x$) represent perfect predictions.

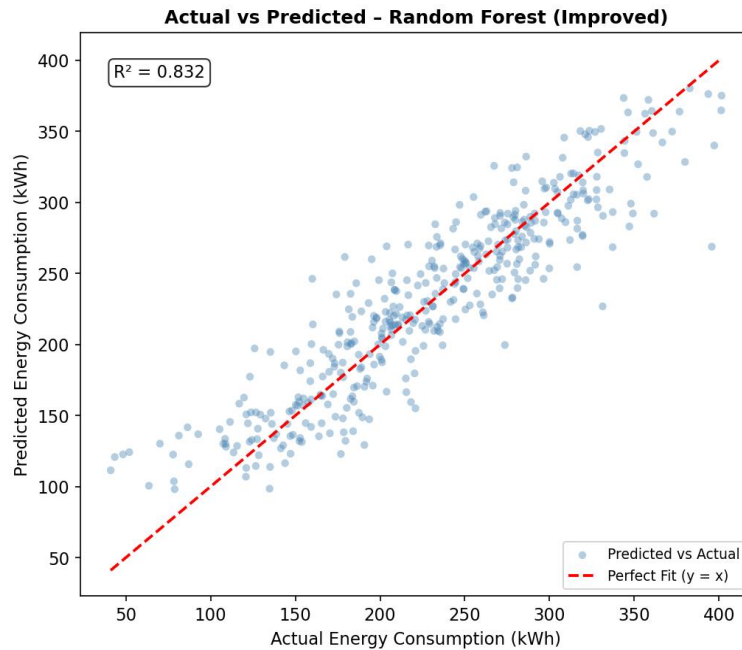


Fig. 4 Actual vs Predicted energy consumption — Random Forest (Improved).

The cloud of points clusters tightly around the diagonal, confirming strong predictive accuracy ($R^2 = 0.832$). A slight fan-shape at high energy values (> 350 kWh) indicates mild heteroscedasticity — the

model is slightly less accurate during peak-consumption events, which are also rarer in the training data.

5.7 Cross-Validation Stability

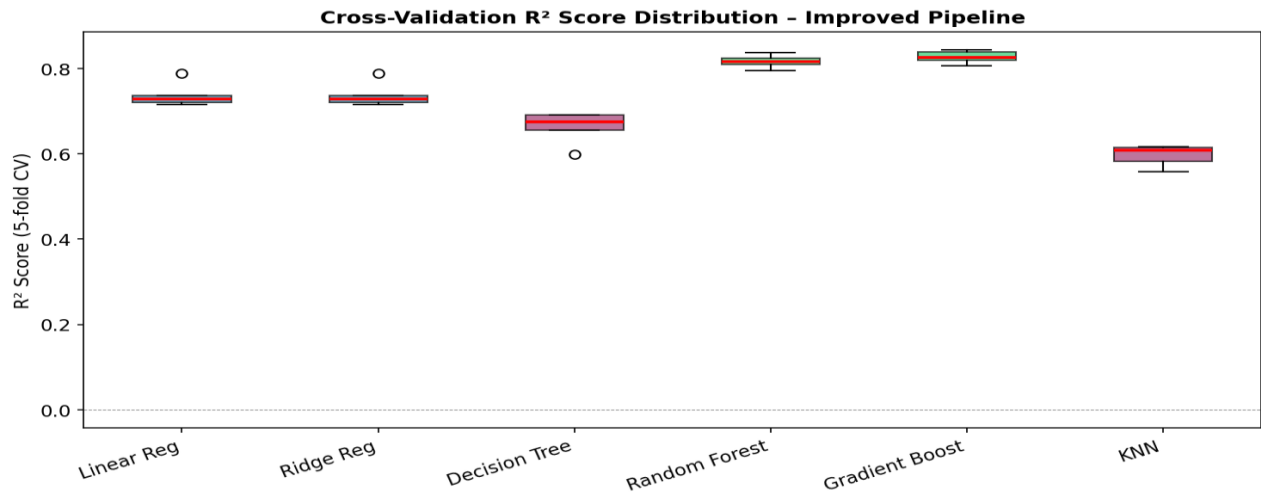


Fig. 5 5-fold CV R^2 distribution across six models (Improved pipeline).

Gradient Boosting and Random Forest show the narrowest interquartile ranges (tight boxes), confirming that their high R^2 scores are stable across different data subsets — not an artefact of a lucky 80/20 split. Linear models show slightly wider variance. Decision Tree shows the widest spread, reflecting its remaining sensitivity to the specific training data subset seen in each fold.

5.8 Feature Importance — Random Forest (Improved)

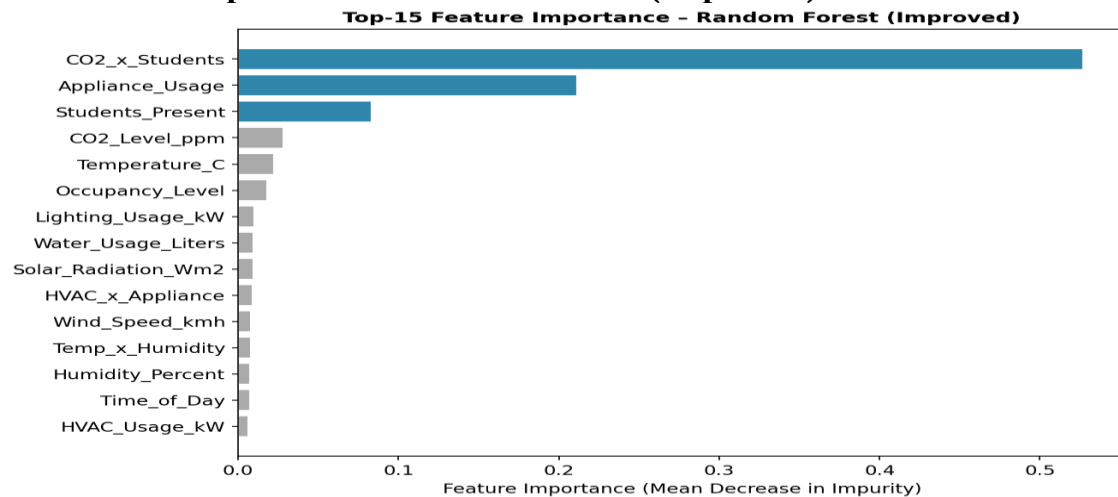


Fig. 6 Top-15 feature importances from Random Forest (Improved pipeline).

HVAC_Usage_kW is by far the most important predictor, consistent with the correlation heatmap (Fig. 2). Lighting_Usage_kW is second. Critically, the engineered feature HVAC_x_Appliance ranks among the top features, validating the feature engineering step. CO2_Level_ppm and Students_Present both appear, confirming that occupancy-related variables carry strong predictive power. Time_of_Day and Temperature_C reflect daily and seasonal cycles. Linear models use all features equally weighted, which is why they cannot exploit this skewed importance distribution as effectively as Random Forest.

6. References

Books:

1. Let Us Python — Yashavant Kanetkar, BPB Publications.
2. Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow (3rd ed.) — Aurélien Géron, O'Reilly Media, 2022.
3. The Elements of Statistical Learning (2nd ed.) — Hastie, Tibshirani, Friedman, Springer, 2009

Online Resources:

- GeeksforGeeks — <https://www.geeksforgeeks.org/machine-learning/>
- Stack Overflow — <https://stackoverflow.com>
- Scikit-learn Documentation — <https://scikit-learn.org/stable/>
- Kaggle Learn — <https://www.kaggle.com/learn>